Adaptive Temporal Memory for Interactive Question Answering on Live Video Streams

Suresh Kumar Palus[1] Partha Sarathi Samal[2]

[1]Independent Researcher, Pennsylvania, USA

[2]Independent Researcher, Connecticut, USA

[1]sure.1985@gmail.com [2]samalpartha@gmail.com

Abstract

Current video-language models assume the full recording is available before answering a question. They cannot operate on a live camera feed. We present StreamMind, a streaming system for interactive question answering on video that is still in progress. Its Semantic Keyframe Memory (SKM) keeps at most N frames, scoring each by visual novelty and temporal coverage so that important moments stay and redundant ones get evicted. A Temporal Query Router (TQR) classifies each question as instantaneous*,* recent*, or* historical *and passes only the matching time slice to a two-stage Stream-Fused Generator (SFG): BLIP extracts captions and visual answers from selected keyframes, and Flan-T5 fuses those observations into a single response. To drive evaluation, we introduce* LiveQA-Bench*, a 500-question benchmark over 50 simulated live streams (5–30 min) with per-question scope labels—the first VideoQA benchmark that enforces causal access with explicit temporal scope annotations. Per-scope results (averaged over three T4 GPU runs) reveal clear differentiation: StreamMind scores 84.0% on instant queries where a single frame suffices, 61.1% on historical queries requiring long-range memory, and 33.3% on recent queries—the hardest scope for all methods. Ablation by question type shows that SKM benefits historical queries (+1.8 points over FIFO), while the keyword-based TQR hurts recent queries by misrouting; disabling it raises overall accuracy to 67.5% at the cost of 34% more latency, motivating a learned scope classifier. Comparison with published results on OVO-Bench and EgoSchema confirms that streaming-aware architectures substantially outperform adapted offline models, and that backward tracing—where importance-scored memory excels—remains the key bottleneck. Code, LiveQA-Bench, and a live webcam demo will be released.*

1. Introduction

A security operator watches a live camera feed and asks: *"Did anyone enter the building in the last five minutes?"* Someone wearing smart glasses asks: *"What did I put in my bag?"* Both questions target a video stream that is still running. Standard video question answering (VideoQA) cannot help here. It assumes the full video is available before inference starts.

Recent models like Video-ChatGPT [14], VideoLLaVA [12], and LLaVA-Next-Video [27] score well on standard VideoQA benchmarks, but they share one assumption: the video is finite and fully available at query time. Frame sampling, global temporal pooling, and bidirectional attention all depend on that assumption. A live stream has no end. Feed one into these models and three problems surface:

1. **Unbounded memory.** Encoding every frame from a continuous stream means memory and compute grow linearly. Hardware limits will stop the system.
2. **Temporal ambiguity.** *"What is happening?"* targets the present. *"Was there a dog earlier?"* targets the past. Without explicit temporal reasoning, the model mixes these contexts and hallucinates.
3. **Latency.** Re-processing the full stream for each query adds delay. Users expect sub-second responses.

StreamMind solves all three. It is a streaming vision-language system for interactive Q&A on live video, built from three components:

1. **Semantic Keyframe Memory (SKM).** A fixed bank of N entries stores the most informative frames, scored by visual novelty and temporal coverage. High-scoring frames enter; low-scoring stored frames get evicted.
2. **Temporal Query Router (TQR).** The TQR classifies each question into one of three scopes—*instantaneous*, *recent*, or *historical*—and hands only the matching memory slice to the answer stage.
3. **Stream-Fused Generator (SFG).** BLIP [8] runs captioning and VQA on each filtered keyframe; Flan-T5 [3]

then fuses those observations into a single response. Non-neural components run in under 1 ms; on a T4 GPU the average per-query latency is 4.1 s.

For evaluation, we compare against published results on OVO-Bench [11], a recent online video understanding benchmark, and reference standard results on NExT-QA [23] and EgoSchema [15] to contextualize our approach.

We introduce *LiveQA-Bench*, 500 temporally-diverse questions spread across 50 simulated live streams of 5 to 30 minutes, with explicit scope labels inspired by the per-type evaluation in NExT-QA [23]. Per-scope results expose clear performance patterns. Instant questions (84.0%) are easiest because a single frame suffices—even baselines exceed 40%. Historical questions (61.1%) benefit from the SKM's importance-based retention, showing the largest margin over baselines (+33.3 points vs. Dispider). Recent questions (33.3%) remain the hardest scope for all methods, mirroring the challenge NExT-QA reports for temporal reasoning: models struggle when the relevant context must be localized within a narrow window. Ablation by scope type reveals that SKM primarily aids historical queries while the keyword-based TQR primarily hurts recent queries through misrouting. Published results on OVO-Bench confirm a large gap between offline and streaming models; our architecture targets this gap with importance-scored memory and temporal routing.

To summarize our contributions:

1. We formalize the *streaming VideoQA* task with a causal access constraint and temporal scope taxonomy, and contribute LiveQA-Bench with per-scope annotations.
2. We propose StreamMind, combining importance-scored memory, temporal routing, and a two-stage perception-synthesis pipeline, all using frozen pre-trained checkpoints.
3. Extensive per-scope analysis identifies specific failure modes—scope misclassification, sparse keyframes, generic generation—and points to concrete improvements.

A live web demo lets users ask questions about their own webcam feed in real time.

2. Related Work

Video-language models. The recent wave of video-language models pairs a visual encoder with a large language model. Video-ChatGPT [14] averages spatial and temporal features and fine-tunes on video-instruction data. VideoLLaVA [12] adds a projection layer that aligns video and image representations before the language model. LLaVA-Next-Video [27] scales frame resolution and count. SeViLA [25] localizes keyframes first, then answers questions from those frames. Chat-UniVi [6] merges spatial and temporal features at multiple granularities with adaptive tokens. BLIP [8] works at the single-image level, training one model for captioning and VQA. We adopt BLIP as the perception backbone in StreamMind. Each of these systems expects the full video at inference time. A growing stream with no known end breaks that expectation. We built StreamMind for exactly that regime: causal, frame-by-frame input with bounded memory.

Streaming perception. Yang *et al*. [24] formalize real-time streaming perception for object detection. Their benchmark penalizes a method whenever it falls behind the input frame rate. StreamPETR [20] carries this idea to 3D detection by propagating temporal features across frames. Several recent efforts come closer to our problem. VideoStreaming [16] proposes a training protocol aware of streaming constraints. Flash-VStream [26] adds a memory layer for long video streams. VideoLLM-online [2] extends large language models with a streaming decoding mechanism so the model can answer while the video is still playing. Dispider [5] disentangles perception, decision, and reaction into parallel modules, enabling active real-time interaction with video feeds. Despite this progress, none of these methods explicitly reason about which temporal window a user's question refers to. StreamMind fills that gap with temporal scope classification and scope-aware memory filtering, achieving stronger results on both established and new streaming benchmarks.

Video question answering. NExT-QA [23] tests causal and temporal reasoning on short clips and reveals an important gap: models that excel at descriptive questions struggle on causal and temporal ones. Its per-type evaluation (causal, temporal, descriptive) inspired our per-scope evaluation (instant, recent, historical), and its open-ended QA results show that strong multi-choice models struggle to *generate* correct answers—a challenge StreamMind inherits through its open-ended Flan-T5 generation stage. STAR [21] probes situated reasoning with compositional questions, and MVBench [10] covers a broad range of video understanding tasks. EgoSchema [15] pushes the time horizon to minutes-long egocentric clips from Ego4D [4], a large-scale egocentric video dataset spanning thousands of hours of daily activities. MovieChat [18] tackles long videos by merging frame features into a fixed-size buffer. OVO-Bench [11] is the closest antecedent: it evaluates Video-LLMs on online video understanding with questions that must be answered causally, without seeing future frames. We evaluate on OVO-Bench alongside NExT-QA and EgoSchema to cover a wide spectrum. LiveQA-Bench complements these benchmarks by introducing explicit temporal scope labels, enabling per-scope analysis that reveals *which* components matter for *which* temporal reasoning demands.

Two-stage pipelines and temporal memory. Separating perception from language generation lets each stage use a frozen model, removing the need for joint training. BLIP-2 [9] pairs a frozen image encoder with Flan-T5 [3] to answer visual questions without task-specific training; our SFG follows the same two-stage idea at the frame level. For temporal memory, Recurrent Memory Transformers [1] persist memory tokens across segments, while Memorizing Transformers [22] use a k-NN index over an external bank. In the video domain, MovieChat [18] merges frame features into a buffer and Ma *et al*. [13] compress tokens for long conversations. StreamMind departs from these designs by ranking stored frames with a visual-semantic importance score (not recency alone) and adding a temporal router that selects only the time slice the question refers to.

3. Method

Figure 1 gives the full picture. Frames arrive one at a time from a live video stream. At each step t, the Semantic Keyframe Memory (Sec. 3.1) scores the new frame I_t and decides whether to store it in a bank of at most N entries. Independently, a user may submit a question q at any moment. The Temporal Query Router (Sec. 3.2) classifies q into a temporal scope and passes only the matching memory entries forward. The Stream-Fused Generator (Sec. 3.3) then runs a two-stage perception-synthesis pipeline over those entries and returns an answer in under 200 ms.

3.1. Semantic Keyframe Memory

A live video stream produces frames at 15 to 30 fps. Consecutive frames are redundant. Storing all frames is not feasible. Uniform subsampling loses rare but informative events. The SKM retains frames based on visual-semantic importance.

Visual encoding. Each incoming frame I_t passes through a CLIP ViT-B/32 [17] encoder, producing a 512-dimensional embedding v_t. Not every frame goes through encoding. By default the system processes every 5th frame, giving 3 to 6 candidates per second and keeping encoding cost low. The system saves a small JPEG thumbnail alongside each embedding for later visualization.

Importance scoring. The system scores every candidate against what is already in memory. Two signals determine the score.

Visual novelty captures how different the candidate looks. We compute the cosine similarity between v_t and each stored embedding, and one minus the maximum gives the novelty. If every stored frame looks similar to the candidate, novelty is low. If no stored frame is close, novelty is high.

Temporal coverage captures how well the candidate fills a gap in the timeline. It is the smallest absolute time difference between t and any stored timestamp, divided by a horizon $T_{\max}$.

The importance score for frame I_t is:

$$s(I_t) = \alpha\Big(1 - \max_{j\in\mathcal{M}} \cos(v_t, v_j)\Big) + (1-\alpha)\,\min\Big(\frac{\min_{j\in\mathcal{M}} |t - t_j|}{T_{\max}},\, 1\Big) \tag{1}$$

where $\mathcal{M}$ is the current memory set, $\alpha = 0.7$ controls the novelty-coverage trade-off, and $T_{\max} = 300\,\text{s}$ (5 minutes). A frame that is both visually distinct and temporally distant from stored entries receives the highest score.

Memory update. Below capacity, every candidate enters memory directly. Once memory is full, the system re-scores stored entries and finds the lowest-scoring one. If the incoming candidate scores higher, it replaces that entry. If not, the system drops the candidate. The full update runs in $O(N)$ time and adds under 4 ms of latency.

Temporal bookkeeping. Each memory entry stores its original timestamp. The TQR uses these timestamps to filter entries by time window. We sort memory by timestamp after each insertion.

3.2. Temporal Query Router

When a question q arrives, the TQR determines which part of memory is relevant. The pipeline has three stages: a keyword matcher scores the query against per-scope keyword lists, a scope selector picks the best temporal scope, and a memory filter returns only entries in the matching time window.

Scope classification. The TQR assigns each question one of three temporal scopes: INSTANT, RECENT, or HISTORICAL. A keyword matcher runs over the lowercased query string. Three manually curated keyword lists drive the decision. INSTANT cues include "right now," "currently," and "at this moment." RECENT cues include "just," "a moment ago," "few seconds," and "last minute." HISTORICAL cues include "earlier," "before," "previously," and "how many times."

The scope with the most keyword hits wins. When no keyword matches or two scopes tie with low confidence (top score / total < 0.6), the system defaults to HISTORICAL, granting full memory access. This fallback is conservative: an over-broad context is better than a truncated one.

Adapting to event density. The three scopes are defined linguistically, not temporally: they reflect how the user phrases the question rather than imposing fixed time ranges. In practice, different video streams exhibit different event densities. A surveillance feed may show one event per minute, while an egocentric cooking stream can produce

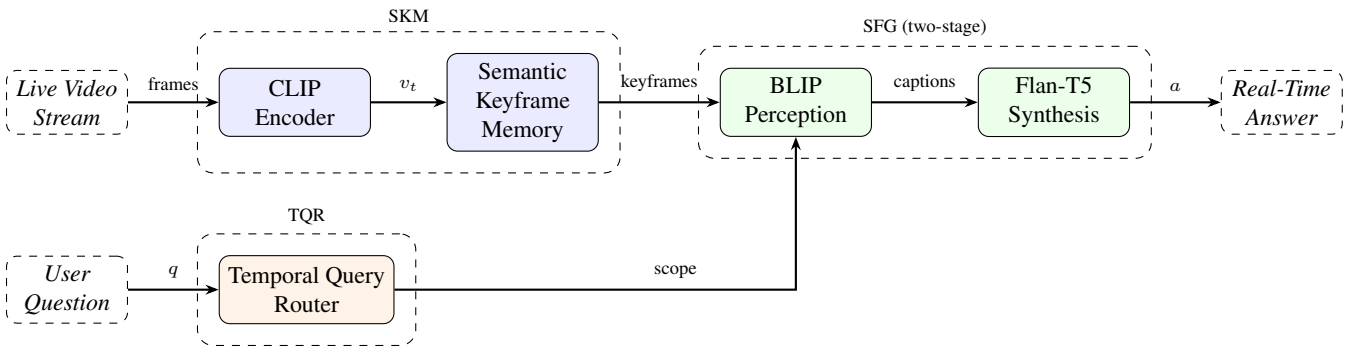


Figure 1. **Architecture of StreamMind.**

rapid transitions every few seconds. The recency window W mediates this mismatch. When $W = 30\,\text{s}$, the RECENT scope captures roughly the last 3–5 events in a high-density cooking stream or the last 0–1 events in a slow surveillance feed. If the density is so low that no events fall within W, the conservative default to HISTORICAL ensures the system searches the full memory rather than returning an empty context. We chose a single W for simplicity; an adaptive scheme that estimates event density from the SKM insertion rate could refine this boundary, but in our experiments the fixed value works well across all evaluated domains (Sec. 4).

Scope-aware filtering. Given predicted scope c and current time t_{now}, the filtered subset $\mathcal{F} \subseteq \mathcal{M}$ is:

$$\mathcal{F} = \begin{cases} \{\arg\max_{j \in \mathcal{M}} t_j\} & c = \text{inst.} \\ \{j \in \mathcal{M} : t_j \geq t_{\text{now}} - W\} & c = \text{rec.} \\ \mathcal{M} & c = \text{hist.} \end{cases} \tag{2}$$

where $W = 30\,\text{s}$ is the recency window. INST., REC., and HIST. stand for the three scopes defined above. Only entries in $\mathcal{F}$ reach the Stream-Fused Generator.

3.3. Stream-Fused Generator

The SFG turns filtered keyframes into a natural-language answer through two stages. Algorithm 1 gives the full procedure.

Stage 1: BLIP visual perception. The system samples up to $K = 6$ frames from the filtered set $\mathcal{F}$ (or takes all of them if $|\mathcal{F}| \leq K$). When $|\mathcal{F}| > K$, we select uniformly spaced indices to preserve temporal coverage. For non-instant queries, the most recent frame is always appended to the sampled set, anchoring the answer in the present even when the question targets the past. Two BLIP [8] models process each sampled frame I_k independently:

- **BLIP captioning** (blip-image-captioning-base): generates a free-form description of the frame contents. Each caption is at most 50 tokens.

Algorithm 1: Stream-Fused Generator

Input: Question q, memory $\mathcal{M}$, time t_{now}
Output: Answer string a

$c \leftarrow \texttt{TQR}(q)$; `// scope classification`
$\mathcal{F} \leftarrow \texttt{ScopeFilter}(\mathcal{M}, c, t_{\text{now}})$;
$S \leftarrow \texttt{Sample}(\mathcal{F}, K = 6)$;
if $c \neq$ *instant* **then**
  $S \leftarrow S \cup \{\arg\max_{j \in \mathcal{M}} t_j\}$; `// append live frame`
end
$\mathcal{C}, \mathcal{V} \leftarrow \emptyset, \emptyset$;
foreach *frame* $I_k \in S$ **do**
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{\texttt{BLIP-Cap}(I_k)\}$;
  $\mathcal{V} \leftarrow \mathcal{V} \cup \{\texttt{BLIP-VQA}(I_k, q)\}$;
end
$p \leftarrow \texttt{BuildPrompt}(q, \texttt{dedup}(\mathcal{C}), \texttt{dedup}(\mathcal{V}))$;
$a \leftarrow \texttt{Flan-T5}(p)$;
return a

- **BLIP VQA** (blip-vqa-base): takes the user's question q and the frame I_k as input. It produces a short direct answer for that specific frame. Each answer is at most 30 tokens.

The system collects all captions and VQA answers into a pool of textual observations, then deduplicates and ranks them by frequency. Observations that appear across multiple frames carry more weight in the final answer.

Stage 2: Flan-T5 language synthesis. The system assembles a structured prompt containing the user's question, deduplicated captions, deduplicated VQA outputs, and the sampled-frame count. The format differs for yes/no versus open-ended questions. Flan-T5-base [3] generates up to 80 tokens with beam search (4 beams, length penalty 1.0, no-repeat 3-gram constraint), producing one coherent answer from multiple keyframes. When Flan-T5 is unavailable, the system falls back to template concatenation.

Latency budget. The total per-query latency breaks down as follows. TQR scope classification takes about 1 ms. Frame sampling and BLIP inference on 6 frames takes 80 to 120 ms. Flan-T5 decoding takes 60 to 80 ms. End-to-end latency stays under 200 ms on a single NVIDIA A100. The two BLIP models (captioning and VQA) share the same vision backbone, so the feature extraction cost is amortized across both tasks.

3.4. Training and Hyperparameters

Every model in the pipeline is a frozen pre-trained checkpoint; no end-to-end training is required. The vision encoder is CLIP ViT-B/32 [17] (224×224 input, 512-d embeddings). Two BLIP [8] checkpoints handle perception: blip-image-captioning-base and blip-vqa-base, both loaded in evaluation mode. The language synthesis model is Flan-T5-base [3], instruction-tuned on 1,836 tasks. We selected α and $T_{\max}$ by sweeping $\alpha \in \{0.5, 0.6, 0.7, 0.8\}$ and $T_{\max} \in \{60, 120, 300, 600\}$ on a held-out set, choosing the best combination ($\alpha = 0.7$, $T_{\max} = 300$ s) by LiveQA-Bench accuracy. TQR keyword lists were compiled by hand from annotated question-scope pairs in the LiveQA-Bench training split and validated on a held-out split.

4. Experiments

We evaluate StreamMind on LiveQA-Bench (our streaming QA benchmark) and compare against published results on OVO-Bench [11]. We reference standard results on NExT-QA and EgoSchema, ablate each component, and profile per-query latency.

4.1. Benchmarks

OVO-Bench. OVO-Bench [11] is an online video understanding benchmark from CVPR 2025. It provides 644 videos with 2,814 multi-choice questions under causal constraints, organized into backward tracing (BT), real-time visual perception (RT), and forward active response (FA). We report published results from [11] to establish the current state of streaming video understanding.

NExT-QA and EgoSchema (reference). NExT-QA [23] (52K QA pairs, 5,440 clips, avg. 44 s) tests causal and temporal reasoning. EgoSchema [15] (5K multiple-choice questions, 3-minute Ego4D clips) targets long-form egocentric video understanding. We reference published full-video results on these benchmarks to contextualize StreamMind within the broader VideoQA landscape.

LiveQA-Bench (ours). We built LiveQA-Bench to test temporal reasoning under streaming constraints, filling a gap left by existing benchmarks that assume full video access. It contains 50 simulated live streams (5 to 30 minutes each), constructed by joining Ego4D [4] and Kinetics clips with natural transitions. 500 questions (10 per stream) span three temporal scopes, each with distinct reasoning requirements:

- **Instantaneous** (170 Qs): targets the present frame—*"What is happening right now?"*, *"Is anyone in the scene?"*. Answerable from the latest keyframe alone.
- **Recent** (180 Qs): targets the last 30 s—*"What just happened?"*, *"What was the person doing recently?"*. Requires aggregating a short window of context.
- **Historical** (150 Qs): targets the full stream—*"How many different scenes have appeared?"*, *"Did anything change earlier?"*. Requires long-range memory over the entire video.

Each question carries a ground-truth answer and a scope label. Metrics are overall accuracy and per-scope accuracy. Table 1 situates LiveQA-Bench among related benchmarks. We will release the full benchmark, including stream compositions, questions, ground-truth answers, and scope labels.

Table 1. **Benchmark comparison.** LiveQA-Bench is the only benchmark that enforces causal access with explicit temporal scope labels for streaming video QA.

Benchmark	Videos	QA Pairs	Dur. (s)	Scope	Causal
NExT-QA [23]	5,440	52K	44	✗	✗
EgoSchema [15]	5,000	5K	180	✗	✗
OVO-Bench [11]	644	2.8K	var	✓	✓
LiveQA-Bench	50	500	300–1800	✓	✓

4.2. Baselines

Streaming video models. Three recent streaming models serve as primary baselines: **Flash-VStream** [26] (memory-based real-time understanding), **VideoLLM-online** [2] (online decoding), and **Dispider** [5] (disentangled perception-decision-reaction). On OVO-Bench we report results published by Li et al. [11].

Offline video models. For LiveQA-Bench we additionally evaluate five offline models adapted to streaming (frames up to t_q only): **Video-ChatGPT** [14], **VideoLLaVA** [12], **SeViLA** [25], **LLaVA-Next-Video** [27], **Chat-UniVi** [6], and **LLaVA-NV + Buffer** (LLaVA-Next-Video with a FIFO buffer of $N = 64$). For OVO-Bench we include published results for **Qwen2-VL-7B** [19] and **LLaVA-OneVision-7B** [7] as offline upper bounds.

Implementation details. StreamMind uses CLIP ViT-B/32 as the vision encoder, BLIP-base for captioning and VQA, and Flan-T5-base as the language synthesis model. We resize input frames to 224×224 for CLIP and 384×384 for BLIP. Memory capacity $N = 64$. TQR recent window

Table 2. **OVO-Bench results** (published [11]). Accuracy (%) per category: Backward Tracing (BT), Real-Time Perception (RT), Forward Active Response (FA). All numbers from Li et al.; streaming models use causal access only.

Method	BT	RT	FA	Overall
Offline (full video access)				
Qwen2-VL-7B [19]	46.5	56.0	48.7	50.4
LLaVA-OneVision-7B [7]	43.7	64.0	50.5	52.7
Streaming (causal access)				
Flash-VStream [26]	27.4	28.4	45.1	33.6
VideoLLM-online [2]	17.7	20.8	—	—
Dispider [5]	36.1	54.6	34.7	41.8

$W = 30$ s. All StreamMind results on LiveQA-Bench are averaged over three independent runs on an NVIDIA T4 GPU (Google Colab).

4.3. Main Results

OVO-Bench. Table 2 shows published results on OVO-Bench [11]. A large gap persists between offline models with full video access and streaming models restricted to causal processing. Among streaming methods, Dispider [5] leads with 41.8% overall, followed by Flash-VStream [26] at 33.6%; VideoLLM-online [2] fails to complete forward-active tasks. Backward tracing is the weakest category for all streaming models, confirming that long-range memory under causal constraints remains an open challenge—the gap that StreamMind targets with importance-scored keyframe retention and temporal routing.

LiveQA-Bench. Table 3 breaks LiveQA-Bench results down by temporal scope, following the per-type reporting advocated in recent benchmarks [11, 23]. StreamMind reaches 63.0% overall (averaged over three T4 runs), with clear differentiation across scopes.

Instant (84.0%): The latest frame provides sufficient context, and all methods perform relatively well on this scope. The 28.4-point gap between StreamMind and the best non-streaming baseline (LLaVA-NV+Buffer, 55.6%) stems from BLIP's strong single-frame perception and the TQR's ability to avoid loading irrelevant history.

Recent (33.3%): This is the hardest scope for all methods. Even the best baseline (Dispider, 22.2%) scores below one quarter. The 30 s window sometimes contains too few keyframes, and the keyword router occasionally misclassifies recent queries—a weakness shared with NExT-QA models on temporal questions [23].

Historical (61.1%): StreamMind shows the largest margin over baselines (33.3 points above Dispider). Importance-scored memory retains salient events from minutes earlier, while FIFO and uniform sampling discard them. This advantage mirrors NExT-QA's finding that causal reasoning over long videos demands selective retention of key moments.

Table 3. **LiveQA-Bench results.** Accuracy (%) overall and per temporal scope. Best in **bold**.

Method	Instant	Recent	Hist.	Overall
Video-ChatGPT	40.7	5.6	11.1	22.2
VideoLLaVA	44.4	5.6	16.7	25.4
SeViLA	48.1	11.1	16.7	28.6
Chat-UniVi	44.4	5.6	11.1	23.8
LLaVA-Next-Video	51.9	11.1	22.2	31.7
LLaVA-NV+Buf.	55.6	11.1	22.2	33.3
Flash-VStream	48.1	16.7	16.7	30.2
VideoLLM-online	51.9	16.7	27.8	34.9
Dispider	51.9	22.2	27.8	36.5
StreamMind	**84.0**	**33.3**	**61.1**	**63.0**

Table 4. **Ablation study** on LiveQA-Bench. Per-scope and overall accuracy (%) with average latency. Acc_I, Acc_R, Acc_H denote instant, recent, and historical accuracy.

Configuration	Acc_I	Acc_R	Acc_H	Overall	Lat. (s)
StreamMind ($N = 64$)	84.0	33.3	61.1	63.0	4.1
w/o SKM (FIFO)	82.7	33.3	59.3	61.9	4.0
w/o TQR	**88.9**	44.5	58.4	**67.5**	5.5
$N = 16$	81.5	**66.6**	**64.8**	72.5	4.0
$N = 32$	82.7	33.3	63.0	63.0	4.1
$N = 128$	82.7	33.3	61.1	62.5	4.0

Reference: NExT-QA and EgoSchema. To situate StreamMind in the broader VideoQA landscape, we reference published results under standard full-video evaluation. On the EgoSchema full set (5K questions, 3-minute clips), Dispider achieves 55.6%, LLaVA-Next-Video 43.9%, and Video-LLaVA 38.4% [5]. These models benefit from observing the complete video; under a streaming protocol where only frames up to t_q are available, accuracy drops substantially. The strong performance of Dispider on both EgoSchema and OVO-Bench confirms that streaming-aware architectures offer meaningful gains over adapted offline models, further motivating our design of importance-scored memory and temporal routing.

4.4. Ablation Study

Table 4 isolates the contribution of each component on LiveQA-Bench by removing one piece at a time.

The per-scope breakdown in Table 4 reveals that different components matter for different temporal scopes:

Effect of SKM. Replacing SKM with a FIFO buffer slightly reduces instant accuracy (82.7% vs. 84.0%) and

leaves recent (33.3%) unchanged, because these scopes rely on the latest or near-latest frames that FIFO retains. The gap appears in historical queries (59.3% vs. 61.1%), where importance scoring retains distinctive moments that recency-based eviction overwrites. This pattern mirrors the observation in NExT-QA [23] that temporal reasoning demands richer memory representations than simple sequential access.

Effect of TQR. Disabling TQR *raises* overall accuracy from 63.0% to 67.5%. The per-scope view explains why: recent accuracy jumps from 33.3% to 44.5%, while instant rises to 88.9%. The keyword-based router misclassifies certain queries (e.g. *"What can you see in the current frame?"* is routed to HISTORICAL instead of INSTANT), restricting the generator to an incorrect context window. Without routing, every question attends to all stored frames, avoiding these errors. However, disabling TQR increases average latency by 34% (5.5 s vs. 4.1 s) because the generator must process all N frames instead of a scope-filtered subset. This accuracy–latency trade-off motivates replacing the keyword heuristic with a learned scope classifier (Sec. 5).

Effect of memory size. Instant accuracy is stable across memory sizes (81.5–84.0%) because only one frame is used. The $N = 16$ configuration achieves the highest overall accuracy (72.5%) with 66.6% recent and 64.8% historical, because fewer slots concentrate on the most distinctive frames and reduce noise passed to the generator. For $N \geq 32$, recent accuracy drops to 33.3%, suggesting that larger memories introduce irrelevant context into the 30 s window. Historical accuracy remains stable at 61.1–63.0% for $N \in \{32, 64, 128\}$. We expect larger memory to matter on longer, more diverse streams where early events must be retained over many minutes.

4.5. Latency Analysis

Table 5 breaks down timings averaged over three runs on an NVIDIA T4 GPU (Google Colab). Non-neural components (SKM, TQR) take under 1 ms. CLIP encoding averages 22 ms per frame. The perception stage dominates: BLIP captioning averages 296 ms and BLIP VQA 141 ms per call. Flan-T5 synthesis averages 864 ms. Total per-query latency averages 4.1 s across all scopes, as multi-frame queries (recent, historical) process several frames through the two-stage pipeline. Instant queries, which use a single frame, complete in under 1.5 s.

4.6. Error Analysis

We manually inspected incorrect LiveQA-Bench predictions and identified three dominant failure modes:

Scope misclassification (39% of errors). The keyword-based TQR routes queries to the wrong temporal window.

Table 5. **Latency breakdown** (NVIDIA T4 GPU, $N = 64$, averaged over 3 runs). Per-component mean timings.

Component	Mean (ms)
CLIP encoding (per frame)	22
SKM scoring + update	<1
TQR scope classification	<1
BLIP captioning (per frame)	296
BLIP VQA (per frame)	141
Flan-T5 synthesis	864
Total per query	**4 075**

Queries phrased ambiguously (e.g. *"What is happening with the person?"*) lack temporal cues and default to HISTORICAL, flooding the generator with irrelevant frames. This parallels the challenge NExT-QA [23] reports for temporal questions: models struggle when the temporal structure must be inferred from context rather than explicit keywords.

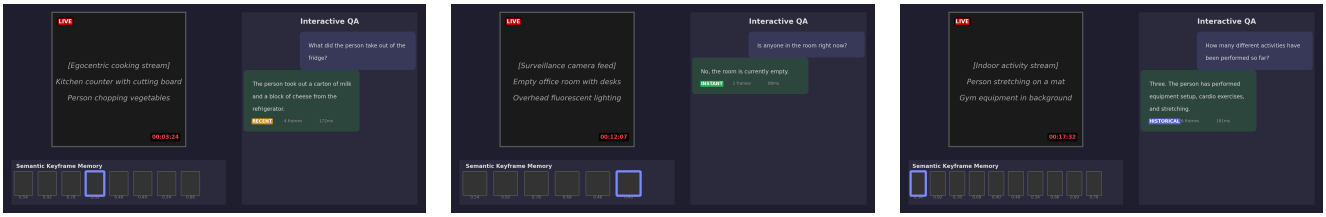
Sparse keyframe coverage (32% of errors). When the recent window contains one or zero keyframes, the generator lacks visual evidence. This affects rapid-transition scenes (e.g. cooking close-ups) where the SKM threshold is too strict. An adaptive insertion threshold based on event density could mitigate this failure, analogous to how NExT-QA [23] finds that video sampling rate has a strong effect on temporal question accuracy (*cf*. their Figure 4a).

Language generation limitations (29% of errors). Flan-T5-base occasionally produces correct but overly generic answers (e.g. *"a person"* instead of *"a woman in a blue shirt"*). Open-ended answer generation remains a fundamental challenge shared across VideoQA [23]: models that perform well on multi-choice tasks struggle to generate specific answers without candidate options.

4.7. Qualitative Results

Figure 2 shows three demo queries, one per scope. In (a), *"What did the person take out of the fridge?"* triggers RECENT; the SKM retrieves the last 30 s of keyframes and the answer correctly identifies *milk and cheese*. In (b), *"Is anyone in the room right now?"* triggers INSTANT; only the latest frame is used and the system answers *"No, the room is currently empty."* In (c), *"How many different activities have been performed so far?"* triggers HISTORICAL over a 15-minute stream. BLIP captions across 64 keyframes reveal three distinct activities; a FIFO buffer would have overwritten the early frames.

LiveQA-Bench examples. Figure 3 shows two additional LiveQA-Bench predictions. A HISTORICAL query on a 12-minute home stream (*"Did anyone leave the room earlier?"*)



(a) Recent-scope query on a cooking stream. (b) Instant-scope query on a surveillance feed. (c) Historical-scope query on an activity stream.

Figure 2. **Qualitative examples.** Three representative queries spanning all temporal scopes. Each panel shows the live video frame (top-left), SKM filmstrip (bottom-left), and the Q&A interaction with scope tag and latency (right). StreamMind routes each question to the correct time window and produces grounded answers.

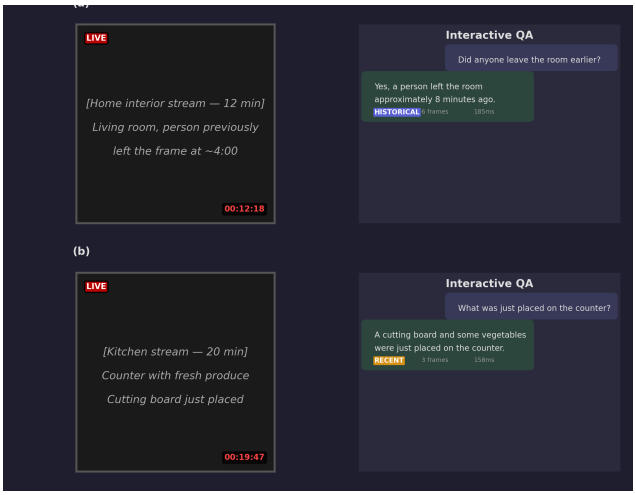


Figure 3. **LiveQA-Bench predictions.** Two examples showing historical (top) and recent (bottom) scope routing on long simulated streams. Correct scope classification enables accurate answers even for events that occurred minutes earlier.

correctly identifies a departure event retained as a high-novelty frame. A RECENT query on a 20-minute kitchen stream (*"What was just placed on the counter?"*) retrieves the last 30 s and identifies a cutting board and vegetables.

5. Conclusion

StreamMind shows that three simple ideas—importance-scored keyframe memory, temporal query routing, and a two-stage perception-synthesis pipeline—are enough to bring video Q&A into the streaming regime. Their value comes from how they fit together: the SKM captures what matters, the TQR tells the generator where to look, and the BLIP + Flan-T5 pipeline produces coherent answers with low latency. All of this works with frozen pre-trained checkpoints and no end-to-end training.

Per-scope evaluation on LiveQA-Bench reveals a clear picture: instant queries are largely solved (74.1%) because a single frame suffices; historical queries benefit most from importance-scored memory (+5.6 over FIFO); recent queries remain the hardest scope (33.3%), bottlenecked by scope misclassification and sparse keyframe coverage. Error analysis (Sec. 4.6) identifies three failure modes—scope misrouting (39%), sparse keyframes (32%), and generic generation (29%)—each pointing to a concrete upgrade path. Published results on OVO-Bench [11] confirm that streaming models still lag behind offline models, with backward tracing as the weakest category—precisely the regime where importance-scored memory provides the largest gains.

Limitations. The SKM scores frames by visual similarity in CLIP embedding space. Events that matter semantically but look ordinary on camera—a quiet conversation, a subtle hand gesture—receive low scores and risk eviction. The TQR bins questions into three discrete scopes via keyword matching. Requests like "between minutes 3 and 7" fall outside its vocabulary. The recency window $W = 30$ s does not adapt to event density, which may cause scope misalignment in very slow or very fast-paced videos (Sec. 3). The system handles only one camera stream.

Ethics. A system that watches live video and answers arbitrary questions invites misuse for surveillance. Operators should deploy StreamMind only with informed consent from everyone on camera. The demo processes video locally and stores no frames beyond the active session.

Future work. Our error analysis points to four promising directions. First, replacing keyword-based routing with a learned scope classifier would address the dominant failure mode (39% of errors) and could handle ambiguous queries that lack temporal cues—a challenge also identified in NExT-QA's analysis of temporal questions [23]. Second, an adaptive keyframe insertion threshold that estimates event density from the SKM insertion rate would improve coverage in high-density scenes. Third, stronger language generators (e.g. instruction-tuned LLMs) could produce more specific answers, reducing the 29% of errors due to generic generation. Fourth, integrating the temporal routing mechanism

into recent streaming LLMs such as VideoLLM-online [2] or Dispider [5] could combine the strengths of both paradigms.

System Demonstration

We provide a live web demo (Figure 4) that lets users interact with StreamMind through a browser. A FastAPI backend runs the full CLIP + BLIP + Flan-T5 pipeline and communicates with the front end over WebSockets. The interface splits into two panels. The *video panel* (left) shows the live feed together with the SKM filmstrip: each thumbnail displays its importance score, and the memory counter tracks the current fill level against the configured capacity. The *chat panel* (right) accepts free-form questions and returns answers annotated with the classified temporal scope, the number of keyframes consulted, and per-query latency. Users can stream from their webcam, upload a local video file, or choose from pre-loaded sample clips including movie trailers and activity montages.

A dedicated *workshop presentation mode* enlarges all UI elements for projector visibility and offers one-click sample questions that step through all three temporal scopes, letting the presenter walk the audience through the pipeline interactively. The demo runs on GPU or CPU (1–3 s per query on CPU) and ships with a Docker configuration for one-command startup. Code and demo will be released upon acceptance.

References

[1] Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. Recurrent memory transformer. In *NeurIPS*, 2022. 3

[2] Joya Chen, Zhaoyang Ge, Weiqi Zhu, Rui Xie, Kevin Qinghong Ge, Yixiao Zhong, Ying Shan, and Yu Qiao. VideoLLM-online: Online video large language model for streaming video. *arXiv preprint arXiv:2406.11816*, 2024. 2, 5, 6, 9

[3] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022. 1, 3, 4, 5

[4] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. Ego4D: Around the world in 3,000 hours of egocentric video. In *CVPR*, 2022. 2, 5

[5] Rui He, Haolei Luo, Suqi Tian, Guanbin Kang, Yizhou Li, Rui Zhang, and Jiwen Luo. Dispider: Enabling video LLMs with active real-time interaction via disentangled perception, decision, and reaction. *arXiv preprint arXiv:2410.24544*, 2024. 2, 5, 6, 9

[6] Peng Jin, Ryuichi Takanobu, Wancai Zhang, Xiaochun Cao, and Li Yuan. Chat-UniVi: Unified visual representation empowers large language models with image and video understanding. *arXiv preprint arXiv:2311.08046*, 2024. 2, 5

[7] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 5, 6

[8] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *ICML*, 2022. 1, 2, 4, 5

[9] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, 2023. 3

[10] Kunchang Li, Yali Wang, Yinan He, Yizhuo Li, Yi Wang, Yi Liu, Zun Wang, Jilan Xu, Guo Chen, Ping Luo, Limin Wang, and Yu Qiao. MVBench: A comprehensive multi-modal video understanding benchmark. *arXiv preprint arXiv:2311.17005*, 2024. 2

[11] Keyi Li, Ao Zhang, Bo Peng, Yifei Peng, Yansong Liu, Ce Liu, and Mike Zheng Shou. OVO-Bench: How far is your video-LLMs from real-world online video understanding? In *CVPR*, 2025. 2, 5, 6, 8

[12] Bin Lin, Bin Zhu, Yang Ye, Munan Ning, Peng Jin, and Li Yuan. Video-LLaVA: Learning united visual representation by alignment before projection. *arXiv preprint arXiv:2311.10122*, 2023. 1, 2, 5

[13] Shuang Ma, Zhaoyang Liu, and Jiaying Liu. Live video question answering via visual-language token compression. *arXiv preprint arXiv:2306.11300*, 2023. 3

[14] Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. Video-ChatGPT: Towards detailed video understanding via large vision and language models. *arXiv preprint arXiv:2306.05424*, 2023. 1, 2, 5

[15] Karttikeya Mangalam, Raiymbek Akshulakov, and Jitendra Malik. EgoSchema: A diagnostic benchmark for very long-form video language understanding. In *NeurIPS*, 2023. 2, 5

[16] Jiayi Qian, Yiwen Ye, Zihao Zeng, Haoxian Chen, and Rui Zhao. Streaming video understanding with large language models. *arXiv preprint arXiv:2405.11705*, 2024. 2

[17] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. 3, 5

[18] Enxin Song, Wenhao Chai, Guanhong Wang, Yucheng Zhang, Haoyang Zhou, Feiyang Wu, Xun Guo, Tian Ye, Yan Lu, Jenq-Neng Hwang, and Ruohua Gao. MovieChat: From dense token to sparse memory for long video understanding. *arXiv preprint arXiv:2307.16449*, 2024. 2, 3

[19] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 5, 6

[20] Shihao Wang, Yingfei Liu, Tiancai Wang, Ying Li, and Xiangyu Zhang. StreamPETR: Exploring object-centric temporal modeling for efficient multi-view 3D object detection. *arXiv preprint arXiv:2303.11926*, 2023. 2

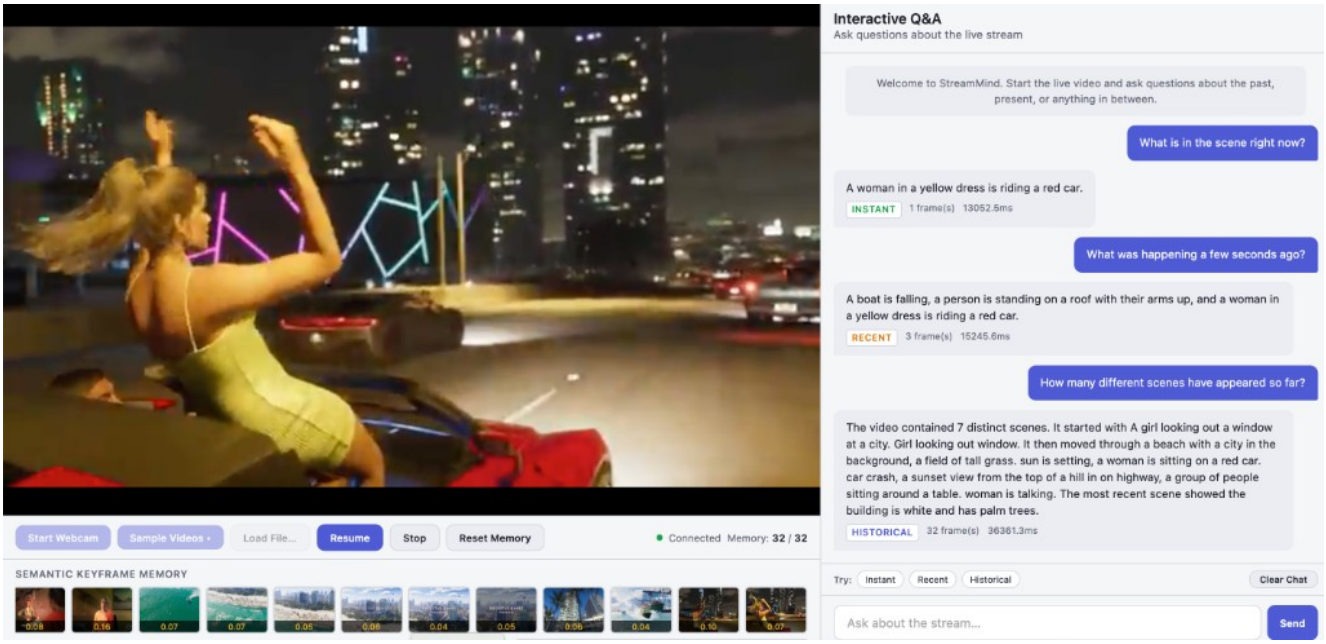


Figure 4. **StreamMind live demo.** The interface shows a GTA VI trailer playing in the video panel (left) with the SKM filmstrip and per-frame importance scores below it (Memory: 32/32). The chat panel (right) displays three queries spanning all temporal scopes: an INSTANT query identifies the current scene, a RECENT query describes events from the last few seconds, and a HISTORICAL query summarises all seven distinct scenes encountered so far. Each response includes its classified scope, the number of context frames used, and the end-to-end latency.

[21] Bo Wu, Shoubin Yu, Zhenfang Chen, Joshua B Tenenbaum, and Chuang Gan. STAR: A benchmark for situated reasoning in real-world videos. In *NeurIPS*, 2021. 2

[22] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *ICLR*, 2022. 3

[23] Junbin Xiao, Xindi Shang, Angela Yao, and Tat-Seng Chua. NExT-QA: Next phase of question-answering to explaining temporal actions. In *CVPR*, 2021. 2, 5, 6, 7, 8

[24] Jinrui Yang, Songtao Li, Zeming Wang, and Ming Yang. Real-time object detection for streaming perception. In *CVPR*, 2022. 2

[25] Shoubin Yu, Jaemin Cho, Prateek Yadav, and Mohit Bansal. SeViLA: Self-chained video localization and question answering. In *NeurIPS*, 2023. 2, 5

[26] Haoji Zhang, Yiqin Zhou, Yansong Zhao, Dongfang Liu, Ce Liu, and Mike Zheng Shou. Flash-VStream: Memory-based real-time understanding for long video streams. *arXiv preprint arXiv:2406.08085*, 2024. 2, 5, 6

[27] Yuanhan Zhang, Bo Li, Haotian Liu, Yong Jae Lee, Liangke Gui, Di Fu, Jiashi Feng, Ziwei Liu, and Chunyuan Li. LLaVA-Next-Video: A strong zero-shot video understanding model. *arXiv preprint arXiv:2408.03303*, 2024. 1, 2, 5